

Chapter 2. Developing on Databricks

Databricks has grown significantly in popularity over the past few years, yet many data scientists and ML engineers still struggle to find an effective way to develop their code within the platform.

Notebooks are first-class citizens in Databricks and are widely used by data scientists during the experimentation phase. They are excellent tools for iterative development, and they have helped popularize interactive Python workflows. However, notebooks were not designed to promote best software engineering practices, and code written during experimentation often requires substantial refactoring before it can be deployed to production.

This refactoring typically involves the following:

- Creating functions, classes, and modules
- Separating tasks into logical components
- Isolating configurations that may evolve over time
- Adding unit tests
- Implementing logging and documentation
- Packaging the project's code

Throughout my career, I have often seen data scientists rely on ML engineers to carry out these steps. This dependency often slows down the path to production, even though much of this rework could be avoided if code were written with production in mind from the outset.

I believe data scientists should be empowered to write production-ready code themselves (while they still can use notebooks), and one of the main goals of this book is to demonstrate how to do this. Despite Databricks' growing adoption, there is still a lack of best practices tailored to its unique development environment.

Developing on Databricks relies heavily on clusters, and you need to understand the impact of cluster selection. Therefore, before diving into development workflows, let's briefly review the different types of clusters that are available in Databricks.

Cluster Configurations on Databricks

I'll start by showing you how to set up an all-purpose cluster for connecting from a local environment or running notebooks within the Databricks workspace. Then, I'll walk you through the serverless compute option, which allows anyone to run the code using [Databricks Free Edition](#). All code examples are compatible with both cluster types. In [Chapter 1](#), I explained when and why you might choose serverless over classic compute and vice versa.

All-Purpose and Jobs Compute Configuration

Before we start developing on Databricks, we need to set up a cluster. All-purpose clusters are mainly used for development, while you should use jobs clusters to run Lakeflow jobs. Cluster configurations for both all-purpose and jobs clusters are similar, so I'll just go over the all-purpose cluster settings. You can set up an all-purpose cluster using an API or Terraform, but the easiest way to view the possible configurations of the cluster is via the user interface (UI) located in the compute tab.

[Figure 2-1](#) shows the recommended configurations of the cluster you can follow through the code accompanying this book. The specific configuration settings you should set are as follows:

Policy

Set this to Personal Compute.

Databricks runtime

Set this to 17.3 LTS.

Preferred node type

Select 16 GB Memory, which is one of the smallest sufficient nodes. It is a single-node cluster without any autoscaling policy.

Terminate after ____ minutes of activity

Set this to 30 minutes to reduce costs.

Tags

Set whatever tags you'll need to make sure development costs can be tracked back to the project.

Create new compute

General

Compute name

Maria Vechtomova's Cluster

Policy

Personal Compute 

Performance

☐ Machine learning 

Databricks runtime

17.3 LTS Scala 2.13, Spark 4.0.0 

Preferred node type

Standard_D4ds_v4 16 GB Memory, 4 Cores 

☒ Terminate after

30 

minutes of inactivity

Tags

Key 	Value 
project	hotel_booking

Figure 2-1. Recommended cluster configuration

Let's go through all the different components of the cluster configurations you can choose from.

Policies and access modes

When creating a cluster, you can choose from the following default policies (unless more policies are created by workspace admins):

Personal Compute

This is ideal for creating single-node compute resources assigned to individual users with minimal setup.

Shared Compute

This is suitable for creating larger, multinode resources shared among users.

Power User Compute

This is an upgraded version of Personal Compute that allows for multinode setup and is designed for distributed single-user workloads.

Unrestricted

This allows full access to all compute configurations when setting up new compute resources.

By default, only workspace admins can create clusters with Shared, Power User, and Unrestricted compute policies. They can grant individual users or user groups access to compute policies. Workspace admins can also create dedicated compute policies, which can be handy if you need to pre-install libraries or restrict cluster creation.

You also need to pay attention to the *access modes*: Dedicated (formerly Single), Standard (formerly Shared), and No Isolation Shared. Different modes have different [limitations](#). For example, you need to put init script locations and wheel locations in ALLOW_LIST to use them in standard access mode. Check out the [documentation](#) for details.

Overall, for ML workflows, I recommend using Dedicated access mode.

Runtime versions

Databricks Runtime is the set of software artifacts that run on Databricks clusters. It defines the environment where your code runs. The `pyt` version must match the Python version of your project (in our case, Python 3.12).

In this book, I am using [Databricks Runtime 17.3 LTS](#), which is, at the moment of writing this book, the latest long-term support (LTS) runtime. For production, it is a common organizational practice to only use LTS runtimes: the support cycle for them is three years. Databricks provides [an overview of LTS runtimes](#) and their end-of-support dates. For instance, the end-of-support date for Databricks Runtime 17.3 LTS is October 2028.

The runtime comes with a pre-set system environment. These are the specifications of the environment for the Databricks Runtime 17.3 LTS:

- Operating system: Ubuntu 24.04.3 LTS
- Java: Zulu17.58+21-CA
- Scala: 2.13.16
- Python: 3.12.3
- R: 4.4.2
- Delta Lake: 4.0.0

In addition to the pre-set environment, runtimes contain pre-installed Python libraries. That does not mean you have to use the exact versions of the libraries in the runtime. You can also install another version of the

library you need, but keep in mind that you are not installing it in a fresh environment, which makes it challenging to reproduce the same environment locally.

Photon acceleration

Photon is Databricks' native vectorized query engine, and it's designed to accelerate Spark SQL workloads and DataFrame API calls. It replaces parts of the traditional Spark execution engine with highly optimized code that takes advantage of modern CPU architectures. While enabling Photon increases the DBU rate per hour, it often reduces total cost per workload because jobs complete significantly faster.

However, Photon only accelerates Spark operations and has no effect on single-node Python libraries. A typical ML training script that uses the scikit-learn library operates on Pandas or Polars DataFrames that are loaded into the driver's memory. Since these libraries bypass Spark entirely, enabling Photon provides no performance benefit for the training step itself.

My advice: enable Photon for data preprocessing tasks that use PySpark (joins, aggregations, and feature transformations at scale), then disable it for compute that runs single-node ML training. This gives you the speed benefits where they matter while avoiding unnecessary DBU costs during training.

Worker and driver nodes

In a single-node setup, you do not have an option to set up worker and driver nodes. You can only choose the size of the single node that acts as both master and worker node. If your task involves running Python code that does not require Spark's distributed computing capabilities, you need a single-node compute. You may also need a large single-node cluster if the data you are working with is large.

If your task involves large-scale data processing or distributed ML, you need a multinode setup. You can choose different worker and driver node sizes, depending on your task, and set autoscaling policies if you expect that the data you work with may increase in size. It is crucial to choose the right size for your nodes and the right amount of worker nodes, and you also need to choose proper autoscaling policies since they have a big impact on costs.

Advanced settings

In the advanced settings of the cluster, you can do the following:

- Set up environment variables (e.g., referencing secrets stored in a secret scope)
- Define init scripts
- Specify a Docker image to initialize your cluster
- Specify a logging location

Cost tracking

You can also track cluster-related costs by applying custom tags to a cluster. These tags are propagated to the underlying cloud provider, and you can use them for cost allocation and reporting. To obtain a complete view of infrastructure spend, you should combine Databricks billing data from system tables with cost data from the cloud provider.

Serverless Compute

When it comes to the all-purpose or jobs compute clusters, users have a lot of freedom to adjust configurations as they wish. But this comes with a cost: it will take time for them to find the perfect cluster configuration for their project (and even different cluster configurations for each task).

You can use serverless compute for both development purposes and running a Databricks job. Even though serverless compute may not be sufficient for machine learning model training pipelines, you could still use it for data preprocessing pipelines.

Unlike all-purpose or job compute, we cannot choose the runtime for serverless compute. Instead, we can curate an environment for dependencies. While the concept of an environment and a runtime are similar, environments are much more lightweight.

Serverless environments

In our project, we use Python 3.12, which means we also need to have an environment that supports that Python version. In this book, we use [serverless environment version 4](#), which comes with the following configurations:

- Operating system: Ubuntu 24.04.2 LTS

- Python: 3.12.3
- Databricks Connect: 17
- Scala: 2.13.16
- JDK: 17

Similar to runtimes, serverless environments come with pre-installed libraries. However, the library versions differ slightly between serverless environment 4 and Databricks Runtime 17.3 LTS. This is something to watch out for if your workflows switch between serverless and classic compute, as subtle version mismatches can cause unexpected behavior. I recommend installing your own versions of the libraries you need, to ensure consistency across environments.

Like runtimes, environments have a support cycle of three years. The end-of-support date for environment 4 is in August 2028. You can check out the [release notes](#) for the full overview.

Cost tracking

While you can track the costs of classic compute clusters with tags, you would use usage policies for serverless compute. Serverless usage policies consist of tags that are applied to any serverless compute activity incurred by users assigned to the policy. These tags are recorded in system tables, so they let you attribute serverless usage to specific budgets.

Usage policies can be created programmatically, or you can create them through the UI (see [Figure 2-2](#)). You can also assign permissions to individual users or user groups (see [Figure 2-3](#)). It's also important to note that if you don't specify a usage policy ID when using serverless resources, the first available policy (in alphabetical order) assigned to the user will be automatically applied.

Edit serverless usage policy

General

Name

Workspaces

The workspaces that this serverless usage policy applies to. If empty, the policy applies to all workspaces.

Tags

Used for attribution. Serverless usage policies only support fixed tags.

Tag

Value



Figure 2-2. Creating a usage policy

Workspace settings > Compute > Serverless usage policies >

mlops_usage_policy

[Edit](#)

Configuration **Permissions** Preview



Permissions control who can edit or use this serverless usage policy. [Learn more](#)

[Grant access](#)

Principal

Permissions

Maria Vechtomova (maria@cauchy.io)

Serverless usage policy: Manager

Figure 2-3. Updating permissions for the usage policy

Project Setup

In this section, I will walk you through the dataset used in the MLOps part of the book and the different ways you can execute the project's code.

The book repository can be found on GitHub at

<https://github.com/mvechtomova/oreilly-mlops-with-databricks>. First of all, you should fork the repository so you can go through the next chapters.

The repository contains the following folders: *mlops* (which covers Chapters [2](#) through [6](#)) and *llmops* (which covers Chapters [7](#) through [9](#)).

For now, we'll talk about the *mlops* folder, which is structured into the following subfolders and files:

notebooks/

This folder contains notebooks used in different chapters.

scripts/

This folder contains Python scripts that are executed as part of the Databricks job.

databricks.yml

This is the file the Databricks job is defined in.

pyproject.toml

This file contains the Python package definition for the project.

project_config.yml

This file contains isolated configurations of the project, such as the location of the data, model hyperparameters, and model features. In other words, it captures the settings we expect to change during the project's lifecycle, to ensure that they remain flexible and not hard-coded into the codebase.

src

This folder contains packaged code for the project.

data

This folder contains the required data for the project.

tests

This folder contains unit tests.

version.txt

This file contains a version number that is used for Python package versioning.

First, let's discuss the dataset in the `data` folder.

The Dataset

In Chapters [2](#) through [6](#), we use the Hotel Booking Cancellation Prediction dataset from Kaggle.^{[1](#)} The dataset contains various features related to bookings, the number of week and weekend nights, the time between arrival and the reservation date, types of meals, whether guests requested

parking, room types, and other features. We will use these features to predict the price of a room per night.

This is a dataset that is close to a real-life scenario. In Chapters [3](#) through [5](#), we will work on the following scenarios:

Model Serving

This is real-time inference in which all the features required for model prediction are provided by the user.

Model Serving with feature lookup

This is real-time inference in which some features come from the user and others are retrieved from a feature store. In this case, the model computes the predicted price per night in real time, using the full feature set.

Feature Serving

This is real-time retrieval in which the predicted price per night is precomputed in a batch process and stored in a feature store. At serving time, the user only provides a booking ID, which is used to fetch the stored prediction.

In the *project_config.yml* file, I specified the target and features that will be used for model training. Our configuration file is relatively simple; for more complex projects with hierarchical configurations, it's worth checking out the [Hydra](#) project.

You'll also notice the `usage_policy_id` parameter. We created the usage policy in the section [“Cost tracking”](#), and I'm including it here so you can pass the usage policy when provisioning serverless resources (such as a Model Serving endpoint). This will allow you to properly attribute and track costs associated with those resources:

```
dev:
  catalog: mlops_dev
  schema: hotel_booking

acc:
  catalog: mlops_acc
  schema: hotel_booking

prd:
  catalog: mlops_prd
  schema: hotel_booking
```

```
parameters:
  learning_rate: 0.05
  n_estimators: 200
  max_depth: 3

num_features:
- number_of_adults
- number_of_children
- number_of_weekend_nights
- number_of_week_nights
- car_parking_space
- special_requests
- lead_time

cat_features:
- type_of_meal
- room_type
- arrival_month
- market_segment_type

target: average_price

usage_policy_id: cdad6f29-a213-4525-9330-e25b90afbe7a
```

Next, the *project_config.yml* file contains model hyperparameters, plus catalogs and schemas used in different environments. You can also update this file to put in your own schema and catalog names if you wish. For now, we'll only focus on the development environment; we'll discuss other environments and deployment strategies in [Chapter 5](#).

We will also get to hyperparameter tuning in [Chapter 3](#), where we'll discuss how we can use MLflow to log all the metrics and parameters and how we got to the parameters specified in the file. It is a common approach to do periodic retraining with the same hyperparameters (as hyperparameter tuning is quite costly) and only perform fine-tuning when there are signs of performance degradation.

Python Tooling

Most people in the industry are convinced that we need to start packaging the code. But what is the right way? What tools should we use?

The Python ecosystem has changed significantly over the last several years, starting around 2016. Before that, everyone was using *setup.py* together with *requirements.txt*, plus the following set of tools:

venv or virtualenv

They used these for virtual environment management. The venv tool is shipped with Python 3, and [virtualenv](#) is a package maintained by Python Packaging Authority (PyPa).

pip

They used this for package management.

setuptools

They used this for build backend.

twine

They used this for package publishing.

In data analytics and data science, *conda* was a popular choice. This was a result of the efforts of an organization called Anaconda that was founded in 2012 by Peter Wang and Travis Oliphant,² with the goal of bringing Python into business data analytics. The conda tool is a multipurpose tool that assists with package, virtual environment, and Python version management. It is worth mentioning that in 2024, Anaconda introduced changes in licensing that forced conda users to look for alternatives.

In 2016, the [PEP 518 standard](#) was introduced, and it specified that build system dependencies should be stored in a *pyproject.toml* file. In 2020, [PEP 621](#) was introduced, and it defined how project metadata should be stored in the *pyproject.toml* file.

Between 2017 and 2020, multiple projects started using *pyproject.toml* to serve multiple purposes and try to eliminate the need to have so many tools. The best-known projects were [hatch](#), [poetry](#),³ and [pdm](#).

In 2023, Rust started taking over the Python ecosystem due to its speed, and another multipurpose tool called Rye was released. In February 2024, another Rust-based tool called [uv](#) developed by Astral entered the market, and it has won the hearts of the Python community since then. Astral also took over the development and maintenance of Rye.

The uv tool is, in my opinion, by far the most intuitive tool to use, and it is blazingly fast. It is therefore the tool I will be using in this book.⁴ You should follow the [official installation instructions](#) to install uv.

The pyproject.toml file

Let's walk through the sections of the *pyproject.toml* file we will use for the MLOps project.

Project section

In the `project` section of the *pyproject.toml* file, we specify the Python version and all the project dependencies:

```
[project]
name = "hotel-booking"
dynamic = ["version"]
description = "Code accompanying MLOps with Databrick O'Reilly book"
requires-python = ">=3.12, <3.13"
dependencies = [
    "mlflow==3.11.1",
    "cffi==1.17.1",
    "cloudpickle==3.1.1",
    "numpy==2.4.0",
    "pandas==2.3.0",
    "pyarrow==22.0.0",
    "databricks-sdk==0.102.0",
    "scikit-learn==1.7.0",
    "lightgbm==4.6.0",
    "scipy==1.16.0",
    "pydantic==2.11.7",
    "loguru==0.7.3",
    "python-dotenv==1.1.1",
    "ray[tune]==2.52.1",
    "optuna==4.4.0",
    "sdv==1.24.0",
    "databricks-feature-engineering==0.14.0",
    "databricks-feature-lookup==1.11.0"
]

[project.optional-dependencies]
dev = ["databricks-connect>=17.3.2, <18",
    "ipykernel>=6.29.5, <7",
    "pip>=25.0.1, <26",
    "pre-commit>=4.1.0, <5",
    "pytest>=8.3.4, <9",
    "pytest-mock>=3.14.0, <4"]

ci = ["databricks-connect>=17.3.2, <18",
    "pre-commit>=4.1.0, <5",
    "pytest>=8.3.4, <9",
    "pytest-mock>=3.14.0, <4"]
```

First of all, it is important to note that we use Python 3.12, which matches the Python version of the Databricks Runtime 17.3 LTS runtime and environment version 4 for serverless.

Secondly, all the package dependencies are pinned to very specific versions. While this may seem beneficial at first glance, it is not considered best practice for packages that are intended to be reused across multiple projects, as strict version pinning can easily lead to dependency conflicts. So for reusable packages, it is generally recommended to define a supported range of Python versions and dependency versions instead. In this case, however, the Python package is not designed for reuse; it simply serves as a convenient way to define and manage project-level dependencies.

When you're installing the package, you don't need to manually specify all dependency versions listed in the *pyproject.toml* file. These dependencies are included in the package metadata and will be installed automatically as part of the package installation process.

Optional dependencies are also defined in the *pyproject.toml* file, and you can install them when you need them for development purposes or for tests to run in a CI pipeline, for example. Note that they will not be installed automatically when the package itself is added as a dependency, as they are not included in package metadata.

Build-system section

The `build-system` section of the *pyproject.toml* file specifies the build backend that is used to package and distribute the project. In this case, we are using `setuptools` as the build backend, and the package version is dynamically determined based on the contents of the *version.txt* file.

While several alternative build backends are available, such as [build backend provided by uv](#), or [Hatchling](#), `setuptools` remains the most mature and widely supported backend in the Python ecosystem:

```
[build-system]
requires = [
    "setuptools>=72.0"
]
build-backend = "setuptools.build_meta"

[tool.setuptools.packages.find]
where = ["src"]
include = ["hotel_booking*"]
```

```
[tool.setuptools.dynamic]
version = { file = "version.txt" }
```

Linting and formatting

For linting and formatting, we use [Ruff](#), which is the fastest tool out there for this purpose. It was developed by Astral and written in Rust. Just like uv, it won the hearts of the Python community and replaced the functionality of many other tools. Configurations for Ruff are specified in the `tool` section of the *pyproject.toml* file:

```
[tool.ruff]
line-length = 90

[tool.ruff.lint]
select = [
    "F",      # pyflakes rules
    "E",      # pycodestyle error rules
    "W",      # pycodestyle warning rules
    "B",      # flake8-bugbear rules
    "I",      # isort rules
    "UP",     # pyupgrade rules
    "SIM",    # flake8-simplify rules
    "ERA",    # eradicate rules
    "C",      # pylint convention rules
    "ANN"     # flake8-annotations rules
]
ignore = [
    "D107", # Missing docstring in `__init__`
    "D203", # incorrect-blank-line-before-class
    "D213", # multi-line-summary-second-line
    "C901", # too complex
    "ERA001", # commented out code
]

[tool.ruff.format]
indent-style = "space"
```

These configurations are also used for [pre-commit](#) hooks that should be installed locally to enforce the rules. Pre-commit hooks are defined in the *.pre-commit-config.yaml* file:

```
exclude: ^tests/resources/
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
```

```

    rev: v6.0.0
    hooks:
      - id: check-added-large-files
      - id: check-json
      - id: check-toml
      - id: check-yaml
      - id: end-of-file-fixer
      - id: trailing-whitespace
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.15.1
  hooks:
    - id: ruff
      args: [--fix, --exit-non-zero-on-fix, --show-fixes]
    - id: ruff-format

```

To make pre-commit hooks execute on every commit, install them with the **pre-commit install** command. To ensure all the files follow the predefined rules, run the **uv run pre-commit run --all-files** command as part of the CI pipeline.

Pytest configurations

The *pyproject.toml* file can also contain some pytest configurations that allow it to run tests without running into errors, and for that reason, we must specify the `testpaths` and `pythonpath` configurations:

```

[tool.pytest.ini_options]
addopts = "-s --no-header --no-summary"
testpaths = [ "tests" ]
pythonpath = [ "." ]

```

Now that we've gone through the *pyproject.toml* file and the dataset, let's take a deep dive into the code.

Project Code: Data Preprocessing

You can find the *chapter2_preprocess_data.py* script in the *notebooks* folder. Note that for the code to run, you must either create a catalog and a schema with the name specified in the *project_config.yml* file under the `dev` key or update the file with the catalog name and schema name you prefer to use instead.

First of all, we create a Spark session and load the project configuration:

```

from pyspark.sql import SparkSession

from hotel_booking.config import ProjectConfig

spark = SparkSession.builder.getOrCreate()

cfg = ProjectConfig.from_yaml("../project_config.yml")
catalog = cfg.catalog
schema = cfg.schema

```

We use the `ProjectConfig` class from the `config` module of our custom `hotel_booking` package. This class is built on [Pydantic](#), which provides robust data validation to ensure that all configuration values match the expected data types.

Beyond basic type checking, Pydantic supports enforcement of additional constraints, such as value ranges, allowed patterns (e.g., regular expressions), and custom validation rules. This helps catch configuration errors early by enforcing these rules at runtime, so misconfigurations will fail fast with clear error messages, instead of causing subtle bugs later in the pipeline.

In the following example, we enforce an opinionated valid value range for hyperparameters by using Pydantic's `Field()` validator. Specifically, we use parameters such as `gt` (greater than) and `le` (less than or equal to) to define acceptable bounds for the field's value:

```

import yaml
from pydantic import BaseModel, ConfigDict, Field

class Parameters(BaseModel):
    """Model training hyperparameters with validation."""

    learning_rate: float = Field(gt=0, le=0.2)
    n_estimators: int = Field(ge=10, le=10000)
    max_depth: int = Field(ge=1, le=50)

class ProjectConfig(BaseModel):
    """Project configuration parameters loaded from project_config.yml."""

    model_config = ConfigDict(protected_namespaces=())

    num_features: list[str]
    cat_features: list[str]

```

```

target: str
catalog: str
schema: str
parameters: Parameters
usage_policy_id: str

@classmethod
def from_yaml(
    cls: "ProjectConfig",
    config_path: str,
    env: str = "dev") -> "ProjectConfig":
    """Load and parse configuration settings from a YAML file.

    :param config_path: Path to the YAML configuration file
    :param env: Environment name to load environment-specific settings
    :return: ProjectConfig instance initialized with parsed configuration
    """
    if env not in ["prd", "acc", "dev"]:
        raise ValueError(
            f"Invalid environment: {env}. Expected 'prd', 'acc', or 'dev'"
        )

    with open(config_path) as f:
        config_dict = yaml.safe_load(f)
    config_dict["catalog"] = config_dict[env]["catalog"]
    config_dict["schema"] = config_dict[env]["schema"]
    for k in ["dev", "acc", "prd"]:
        config_dict.pop(k)
    return cls(**config_dict)

```

Before we proceed, we need to make sure the catalog and schema exist and, if not, create them:

```

spark.sql(f"CREATE CATALOG IF NOT EXISTS {catalog}")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog}.{schema}")

```

After we create the Spark session, load the configuration, and make sure catalog and schema exist, we load and clean the data (note the dataset-specific cleaning commands) and store the result in a Delta table.⁵ Also note that we enable `ChangeDataFeed` to track all row-level changes between versions of the Delta table:

```

from datetime import datetime

import pandas as pd

```

```

df = pd.read_csv("../data/booking.csv")
df.columns = df.columns.str.replace(r"[-]", "_", regex=True)
df["date_of_reservation"] = df["date_of_reservation"].apply(
    lambda x: "3/1/2018" if x == "2018-2-29" else x
)
df["date_of_reservation"] = df["date_of_reservation"].apply(
    lambda x: datetime.strptime(x, "%m/%d/%Y")
)
df["arrival_date"] = df["date_of_reservation"] + pd.to_timedelta(
    df["lead_time"], unit="d"
)
df["arrival_month"] = df["arrival_date"].dt.month

dst_table = f"{catalog}.{schema}.hotel_booking"

spark.createDataFrame(df).write.mode("overwrite").saveAsTable(f"{dst_table}")

spark.sql(
    f"""ALTER TABLE {dst_table}
    SET TBLPROPERTIES (delta.enableChangeDataFeed = true);"""
)

```

Now that we've gone through the code (the full version is available in the notebook), our next step is to run it. Before that, however, it is worth taking a closer look at the Databricks developer tools.

Databricks Developer Tools

There are four main local developer tools Databricks provides:

Databricks CLI, Declarative Automation Bundles (DABs), Databricks Connect, and Databricks extension for Visual Studio Code. Let's go through them one by one.

Databricks CLI

Databricks CLI is a command line interface that allows you to interact with the Databricks platform. It is a very powerful tool with a large range of commands that cover functionality that's similar to that of Databricks Terraform modules and APIs.

You should use Databricks CLI to authenticate toward Databricks from your local machine. We need to start by installing it, and Databricks provides [comprehensive documentation](#) on how to do it on different OS.

Databricks CLI provides multiple authentication options. I do not recommend using personal access tokens, as a leaked token could grant full access to whatever resources you have access to across your Databricks workspace and Unity Catalog, including data, models, jobs, and serving endpoints. Instead, use *user-to-machine authentication*,⁶ which relies on short-lived OAuth tokens that expire automatically and do not carry the same risk if intercepted.

For the workspace-level commands, use the following command to authenticate:

```
databricks auth login --host workspace-url
```

It creates an authentication profile in the *.databrickscfg* file that contains the profile name and configurations for the profile. It is also a good practice to use a user-understandable profile name, so that you can pass the *-p* option when authenticating:

```
[mlops-book]
host    = https://oreil.ly/DtiuJ
auth_type = databricks-cli
```

You use the profiles to authenticate with Databricks when you're using developer tools.

Declarative Automation Bundles

You may have noticed that the repository contains the *databricks.yml* file. This is the DAB configuration file, which must be located in the root of the bundle project folder.

At their core, DABs enable you to manage Databricks projects through an IaC approach. Using the *databricks.yml* file, you can define the artifacts and resources your project requires. In [Chapter 5](#), we'll take a deep dive into specific resources you can deploy using DABs. For now, let's explore the syntax and core commands.

Syntax of the *databricks.yml* file

First, every *databricks.yml* file must contain a `bundle` section with a name. Optionally, you can specify the Databricks CLI version and information about your source code. You can also provide an interactive `cluster_id`, which will override any other cluster specifications in the

bundle, but this will only work in development mode (which we'll discuss below). I recommend specifying the `cluster_id` under the target section for development purposes instead, as follows (note that the `cluster_id` specification is not relevant for serverless compute):

```
bundle: # Required.
  name: string # Required.
  databricks_cli_version: string
  cluster_id: string
  git:
    origin_url: string
    branch: string
```

The `artifacts` section is where you can specify which packages need to be built as part of the bundle deployment. It can run any build command (for example, `uv build`) given that the dependency (here, `uv`) is installed.

For the `whl` type, you can set the `dynamic_version` parameter. If you set it to `true`, a dynamic version with a timestamp will be created. This is an important consideration for jobs that use an interactive cluster, as it ensures that the cluster will use the latest code changes. However, this isn't necessary for job clusters or serverless clusters, as they handle this behavior automatically:

```
artifacts:
  some-unique-artifact-identifier:
    build: string
    files:
      - source: string
    path: string
    type: string
    dynamic_version: boolean
```

You can also set up *custom variables* directly in the `databricks.yml` file. Then, you can pass these variables to the deployment by entering the **`databricks bundle deploy --var`** command. For traceability purposes, it is a good idea to pass the `git_sha` variable. We will use this later to tag our experiment runs and models, to ensure that we can always link a model back to the exact version of the code that created it.

You define these variables within a `variables` section like this:

```
variables:
  some-unique-variable-name:
    description: string
    default: string or complex
```

To keep your *databricks.yml* file clean and maintainable, especially as your project grows, you can move job definitions into separate files and reference them using the `include` section. This modular approach helps you avoid a single, monolithic configuration file that becomes hard to read and review:

```
include:
  - 'some-file-or-path-glob-to-include'
  - 'another-file-or-path-glob-to-include'
```

In the `sync` section of the *databricks.yml* file, you can specify what files or directories should be included or excluded from the bundle deployment:

```
sync:
  include:
    - 'some-file-or-path-glob-to-include'
    - 'another-file-or-path-glob-to-include'
  exclude:
    - 'some-file-or-path-glob-to-exclude'
    - 'another-file-or-path-glob-to-exclude'
  paths:
    - 'some-file-or-path-to-synchronize'
```

Next, in the `resources` section of the *databricks.yml* file, you can define all the resources that should be deployed as part of the bundle. DABs support a wide and continuously expanding set of resource types, including alerts, apps, clusters, dashboards, Lakeflow jobs, and SQL warehouses.⁷

This is how you define resources in the *databricks.yml* file:

```
resources:
  clusters:
    some-unique-programmatic-identifier-for-this-cluster:
      # See the REST API create request payload reference for clusters.
  experiments:
    some-unique-programmatic-identifier-for-this-experiment:
      # See the REST API create request payload reference for experiments.
```

```

apps:
  some-unique-programmatic-identifier-for-this-app:
    # See the REST API create request payload reference for apps.
jobs:
  some-unique-programmatic-identifier-for-this-job:
    # See the REST API create request payload reference for jobs.
pipelines:
  some-unique-programmatic-identifier-for-this-pipeline:
    # See the REST API create request payload reference for
    # Lakeflow declarative pipelines.

```

In the `permissions` section, you can specify permissions for all the resources defined in the bundle configuration. You can set permissions for groups, users, or service principals. The allowed top-level permission levels are `CAN_VIEW`, `CAN_MANAGE`, and `CAN_RUN`. You can also specify permissions on the resource level:

```

permissions:
- level: <permission-level>
  group_name: <unique-group-name>
- level: <permission-level>
  user_name: <unique-user-name>
- level: <permission-level>
  service_principal_name: <unique-principal-name>

```

The `run_as` section allows you to specify the identity, such as a service principal, to use to run a job. The user must have `CAN_USE` permissions for the service principal to use this feature:

```

run_as:
  service_principal_name: <service-principal-name>

```

Finally, you can use the `targets` section to override settings that are specified elsewhere in the `databricks.yml` file. For development purposes, it is convenient to specify an interactive `cluster_id` under the `dev` target and utilize the development mode.

The development mode enables the following behavior:⁸

- It adds the `[dev ${workspace.current_user.short_name}]` prefix to all resource names, and it tags each job with a `dev` Databricks tag.

- It allows you to override job clusters (which is not allowed in production mode).
- It pauses all schedules and triggers.
- It enables concurrent runs on all deployed jobs and disables the deployment lock for faster iteration.

This is how you can define targets in the *databricks.yml* file:

```
targets:
  some-unique-programmatic-identifier-for-this-target:
    artifacts:
      # See the preceding "artifacts" syntax.
    bundle:
      # See the preceding "bundle" syntax.
    cluster_id: string
    default: true | false
    mode: development
    presets:
      preset: value
    resources:
      # See the preceding "resources" syntax.
    sync:
      # See the preceding "sync" syntax.
    variables:
      preceding-unique-variable-name: non-default-value
    workspace:
      # See the preceding "workspace" syntax.
    run_as:
      # See the preceding "run_as" syntax.
```

It is common practice to define the *workspace configuration* under a target because workspace-related values, such as the host and authentication details, typically differ by environment.⁹ The `workspace` section can contain the following information:

```
workspace:
  artifact_path: string
  auth_type: string
  file_path: string
  host: string
  profile: string
  root_path: string
  state_path: string
```

Note that in development mode, the bundle's root path must be under the user's home directory, so it is common to have different `root_path` values for different targets.

Now that we've covered the different sections of the *databricks.yml* file, let's take a closer look at the version used in this book's code repository.

The databricks.yml file

In the *databricks.yml* file, we specify the bundle name first:

```
bundle:
  name: oreilly-mlops-with-databricks
```

Then, we define the artifacts to build. In this case, we use the `uv build` command to create the package wheel, which we can reference later as the task library. The path is `."`, which points to the root of the directory:

```
artifacts:
  default:
    type: whl
    build: uv build
    path: .
```

If you have multiple packages within the same repository, the path will be a folder that contains the *pyproject.toml* file.

Next, we define the following bundle variables:

git_sha and branch

We will use these later as arguments for our Python tasks, so that we can tag experiment runs and registered models with actual `git_sha` and `branch` values.

schedule_pause_status

We can use this to pause the scheduled job when the target is not set to development mode, which is convenient if we use the `acc` target.

usage_policy_id

We can use this to pass to serverless resources deployed through the bundle for cost allocation. We will discuss this in detail in

The following example shows how to deploy these bundle variables:

```
variables:
  git_sha:
    description: git_sha
    default: abcd
  branch:
    description: branch
    default: main
  schedule_pause_status:
    description: schedule pause status
    default: PAUSED
# matches project_config.yml
  usage_policy_id:
    description: usage/budget policy id for cost attribution
    default: cdad6f29-a213-4525-9330-e25b90afbe7a
```

In the `sync` section, we specify the `dist` folder to make sure the built artifacts are easily accessible when running the code on Databricks during the development phase:

```
sync:
  include:
    - dist/*
```

Note that we haven't defined any resources directly in our *databricks.yml* file. Instead, we've created separate YAML files for our jobs and placed them in a `resources` folder. Here's how you can include those resources in the bundle definition; make sure there are no irrelevant files in that directory:

```
include:
  - resources/*
```

Finally, in the `targets` section, we define three targets: `dev`, `acc`, and `prd`. The `dev` target is our default and is used primarily for development. It's configured to use development mode, which automatically adds the `dev ${workspace.current_user.short_name}` prefix to all deployed resources and pauses any schedules.

The `acc` target is intended for acceptance testing, and `prd` is used for deploying code to production. The following example shows how to deploy these three targets:

```
targets:
  dev:
    default: true
    mode: development
    workspace:
      host: <your_host>
      root_path: /Workspace/Users/${workspace.current_user.userName}/
                .bundle/${bundle.target}/${bundle.name}
  acc:
    workspace:
      host: <your_host>
      root_path: /Shared/.bundle/${bundle.target}/${bundle.name}
    variables:
      schedule_pause_status: PAUSED
  prd:
    workspace:
      host: <your_host>
      root_path: /Shared/.bundle/${bundle.target}/${bundle.name}
    variables:
      schedule_pause_status: UNPAUSED
```

In [Chapter 5](#), we'll take a closer look at these environments and explore how to promote code among them.

Now that we've gone through the *databricks.yml* file, let's take a closer look at how we can interact with automation bundles.

Using DABs

Before we go into details, you need to make sure that the host defined in the target matches the Databricks profile you use to log in to Databricks. If you've forked the repository and cloned it to your local machine, make sure to update the host value in the *databricks.yml* file. Without this change, you won't be able to run the code examples covered in the following sections.

There are four main Databricks CLI commands related to DABs:

```
databricks bundle validate
```

This checks whether your *databricks.yml* file is configured correctly.

```
databricks bundle deploy
```

This deploys the resources defined in your *databricks.yml* file.

```
databricks bundle run <job resource name>
```

This executes the specified job.

```
databricks bundle destroy
```

This removes all the assets that were created by the bundle.

After the `databricks bundle deploy` command is executed, all the project files and built artifacts are uploaded to the `root_path` location specified under target. There, you'll find the following:

The artifacts folder

This holds the Python wheel files that were built for the project.

The files folder

This contains all your project's source files.

The state folder

This stores the Terraform state of the bundle.^{[10](#)}

The resources folder

This contains the direct deployment state file.

It's important to pay attention to authentication when using DABs, specifically the evaluation order:

1. For any command run from the bundle working directory, evaluate and authenticate the values of fields within a project's bundle setting files.
2. Evaluate and authenticate the values of environment variables (`DATABRICKS_HOST` and `DATABRICKS_TOKEN`).
3. Evaluate and authenticate the configuration profile field values within the *.databrickscfg* file.

Note that if you happen to have an environment variable set up, it may mess up authentication.

Databricks Connect

[Databricks Connect](#) is a Python package that allows you to trigger an execution of PySpark code on a Databricks cluster from a local environment.

Before we proceed, you need to make sure that the host mentioned in the profile configuration has an active all-purpose cluster that matches the specifications we mentioned earlier, when we went through the all-purpose cluster configuration. Alternatively, you can use serverless.

We have already gone through the *pyproject.toml* setup and uv. Now, you need to create a virtual environment that you can activate later. To do this, from the *project* folder in a terminal, you can run the `uv sync` command as follows:

```
uv sync --extra dev #You can also run an alternative command if you use Windows

source .venv/bin/activate

python
```

Note that you need to use Python 3.12 and Databricks Connect version 17.3.x, to match the runtime. With the Python command, you get into the Python console, and then, you can run the following commands using a serverless cluster:

```
from databricks.connect import DatabricksSession

spark = DatabricksSession.builder.profile(
    "mlops-book").serverless(True).getOrCreate()

df = spark.read.table("samples.nyctaxi.trips")

df.show(5)
```

Alternatively, you can use an interactive cluster by providing the cluster ID.

Spark commands will run on a Databricks cluster.

There are a couple of things to pay attention to here:

- You need to choose the correct profile from the `.databrickscfg` file. If the profile is named `DEFAULT`, it is not needed.
- You'll use `DatabricksSession` to run Spark code. If you, for example, run `df.toPandas()`, which you may need for an ML library you are using, the dataframe will be loaded into your machine's memory. And if the data is too big, you will get out-of-memory errors.

It may be surprising, but you can import PySpark, even though PySpark is not listed as a dependency of Databricks Connect and `uv` does not show that PySpark is installed. You can see this if you try to install Databricks Connect in a clean virtual environment as follows:

```
(test) mariavechtomova@Mac test % uv pip install databricks-connect==17.3.5
Resolved 30 packages in 293ms
Prepared 8 packages in 593ms
Installed 30 packages in 301ms
+ certifi==2026.1.4|
+ cffi==2.0.0
+ charset-normalizer==3.4.4|
+ cryptography==46.0.5|
+ databricks-connect==17.3.5|
+ databricks-sdk==0.89.0|
+ google-auth==2.48.0
+ googleapis-common-protos==1.72.0|
+ grpcio==1.78.0|
+ grpcio-status==1.78.0|
+ idna==3.11|
+ numpy==2.4.2
+ packaging==26.0
+ pandas==2.3.3
+ protobuf==6.33.5
+ Py4J==0.10.9.9
+ pyarrow==23.0.1
+ pyasn1==0.6.2
+ pyasn1-modules==0.4.2|
+ pycparser==3.0
+ python-dateutil==2.9.0.post@
+ pytz==2025.2
+ requests==2.32.5
+ rsa==4.9.1|
+ setuptools==82.0.0
+ six==1.17.0
+ typing-extensions==4.15.0|
+ tzdata==2025.3|
+ urllib3==2.6.3
+ zstandard==0.25.0
```

This is because PySpark is “hidden” and embedded into Databricks Connect—which is why it’s a bad idea to list both Databricks Connect and PySpark in your project dependencies. You can view PySpark in this “hidden” location by going to `.venv/lib/python3.12/site-packages` as follows:

```
(test) mariavechtomova@Mac site-packages % ls -la | grep pyspark
drwxr-xI-x@ 56 mariavechtomova staff      1792 Feb 18 00:23 pyspark
```

If you try to use `SparkSession` instead of `DatabricksSession`, however, it would not work because it misses necessary context, as shown here:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()
df = spark.read.table("samples.nyctaxi.trips")
df.show(5)
```

In this case, `import` works without issues because PySpark is part of Databricks Connect. However, when you try to run the second line, you will get the following error message:

```
RuntimeError: "Only remote Spark sessions using Databricks Connect are supported.
Use DatabricksSession.builder to create a remote Spark session instead."
```

The same command does work in the Databricks extension for Visual Studio Code, however, because the extension automatically injects the required connection details through a `.databricks.env` file it creates inside the `.databricks` folder of your project. It contains the following environment variables:

- `DATABRICKS_AUTH_TYPE`
- `DATABRICKS_BUNDLE_TARGET`
- `DATABRICKS_HOST`
- `DATABRICKS_METADATA_SERVICE_URL`
- `DATABRICKS_PROJECT_ROOT`
- `DATABRICKS_SERVERLESS_COMPUTE_ID`
- `PROFILE`
- `PYDEV_D_WARN_SLOW_RESOLVE_TIMEOUT`
- `SPARK_CONNECT_PROGRESS_BAR_ENABLED`

Databricks Extension for Visual Studio Code

The Databricks extension for Visual Studio Code integrates with all other developer tools: Databricks CLI, DABs, and Databricks Connect.¹¹ This makes it very easy to connect to the Databricks workspace and execute your code using a Databricks cluster.

Since this is a monolithic repository (monorepo) setup, I highly recommend installing the Python Envoy VS Code extension,¹² which will automatically detect the virtual environment based on the file you have opened.

After you've installed the extension, follow these steps:

1. Open the project; make sure to choose the *mlops* folder of the repository as the local folder, as shown in [Figure 2-4](#)).
2. Run the `cd mlops & uv sync --extra dev` command to sync dependencies and make sure that the `.venv` virtual environment has been created.
3. Select an interpreter using Python extension (or it will be automatically selected if Python Envoy VS Code extension is installed)
4. Make sure the *databricks.yml* file in the *mlops* folder contains the host that matches the host of the profile that was created earlier.
5. Authenticate using the same profile.
6. Select the cluster that you created earlier or choose the serverless compute option.

In the end, Remote Folder must be present under CONFIGURATION and the Active Environment and Databricks Connect sections must have checkmarks next to them (see [Figure 2-4](#)).

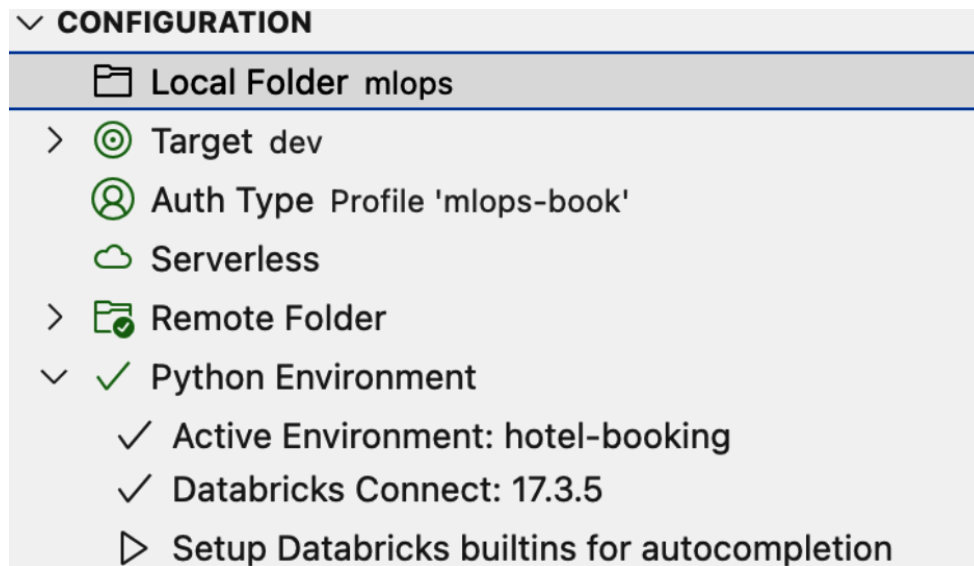


Figure 2-4. Databricks extension in the correct state

Now, let's create a *demo.py* file (the same code we tried to run from a terminal) and add `# Databricks notebook source` so that the code can be recognized as a notebook (see [Figure 2-5](#)). The cells are delimited by a line containing `# COMMAND -----`.¹³

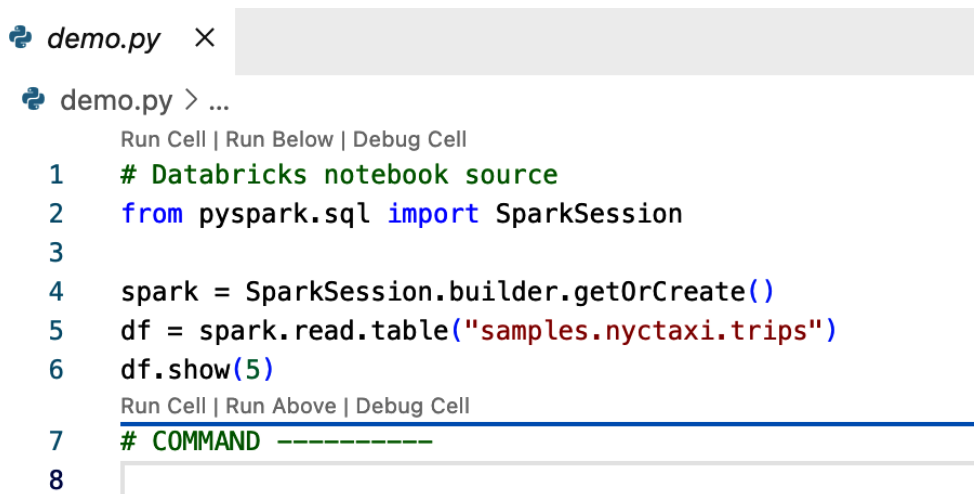


Figure 2-5. Databricks notebook view in Visual Studio code

We can run the code interactively by clicking **Run Cell**. We can also execute project code (for example, *notebooks/chapter2_preprocess_data.py*) from VS Code in a similar way.

However, some code can only run in a Databricks environment. For example, the `databricks-feature-engineering` package is not supported locally, and some Delta table commands will fail outside Databricks. Secondly, even if you use PySpark for preprocessing, most ML libraries require Pandas (or sometimes Polars) DataFrames as input for training. Both Pandas and Polars compute on a single node, which means that even in a Databricks environment, you are not directly leveraging the distributed power of Spark. The advantage of using them, however, is

that you can attach your notebook to a larger cluster with more memory. If you run the same code locally, it will use your laptop's memory, and you can't easily scale that up.

Let's explore how to make notebooks available in the Databricks workspace and execute them there instead.

The Databricks extension for Visual Studio Code allows you to sync the entire project to the workspace. To do this, click the **Sync** button in the Remote Folder tab on the screen shown previously in [Figure 2-4](#), and the extension will run the `databricks sync <folder> /Users/<user_name>/<folder> --watch` command in the background. As a result, any local changes you make will be automatically reflected in the workspace without you having to perform manual synchronization.

You can also use this approach to make a wheel file for your project available in the workspace. Simply run the **uv build** command, and the wheel you generate will be synced along with the rest of the files (unless you add the wheel location to the `.gitignore` file).

It is important to note that the `databricks sync` command provides one-way synchronization. Changes made directly in the Databricks workspace are not reflected locally, and any subsequent sync will overwrite workspace code.

Alternatively, you can make the code accessible in the workspace by using the Git folder functionality. After creating a Git folder, you can authenticate by linking your Git account to Databricks or by using a personal access token from GitHub or another supported version control provider. Git folders enable bidirectional synchronization, which means changes made in notebooks can be pushed back to Git (although this may become inconvenient when dealing with merge conflicts). In addition, managing project dependencies through Git Folders can be challenging.

Finally, you can use the **databricks bundle deploy** command to make all project files available in the workspace together with the built artifact, which is the main advantage this approach has over syncing. (Make sure to run the command from the `mlops` folder that contains the `databricks.yml` file.) However, this method also provides one-way synchronization. Note that using the `sync` command together with `bundle deploy` may corrupt the bundle state.

Running Code in a Databricks Notebook

You may have noticed that the *chapter2_preprocess_data.py* script is also just a Python file (not an *IPython* notebook with the *.ipynb* extension) that has `#Databricks notebook source` as the first line and that the cells are delimited by `# COMMAND -----`.

Let's assume we used the `databricks bundle deploy` command, which makes this notebook available under the `files` folder in the bundle root in the workspace. [Figure 2-6](#) shows what the notebook looks like if we open it.

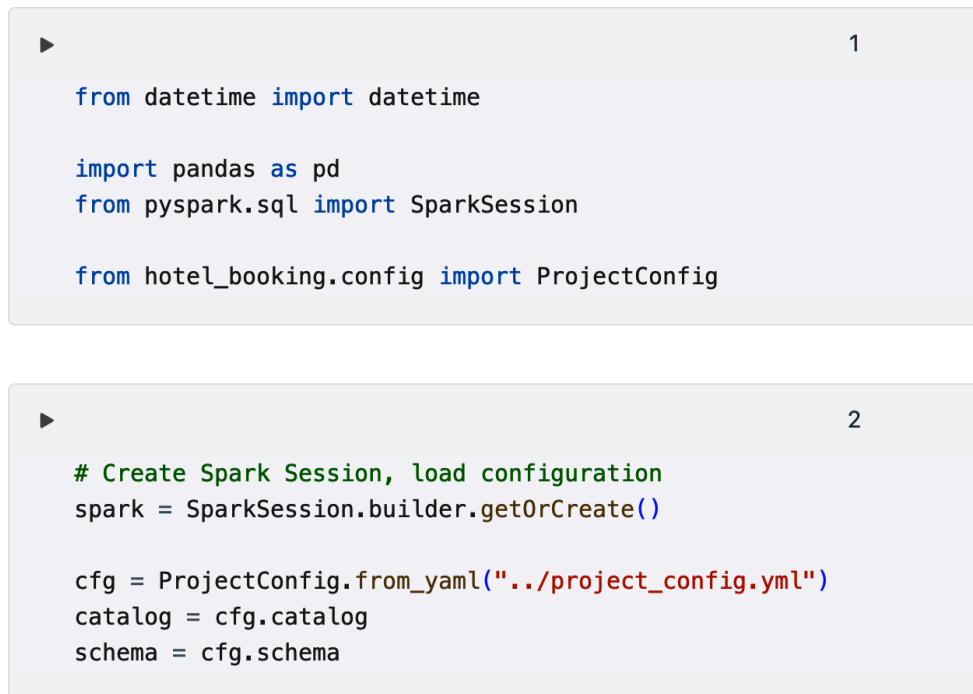


Figure 2-6. Databricks notebook view in the Databricks workspace

To run a Databricks notebook, you need to select a cluster and install the project package. Let's see how you can do this for serverless and all-purpose compute.

Using serverless compute

When using serverless compute, you must choose an environment before proceeding. In the example in [Figure 2-7](#), we select a Base environment of Standard v4 and install the custom package wheel that was built together with the bundle (which is available under the bundle root in the *artifacts/internal* folder). Notice that we selected the `mlops_usage_policy` here so that the costs related to project development will be attributed correctly.

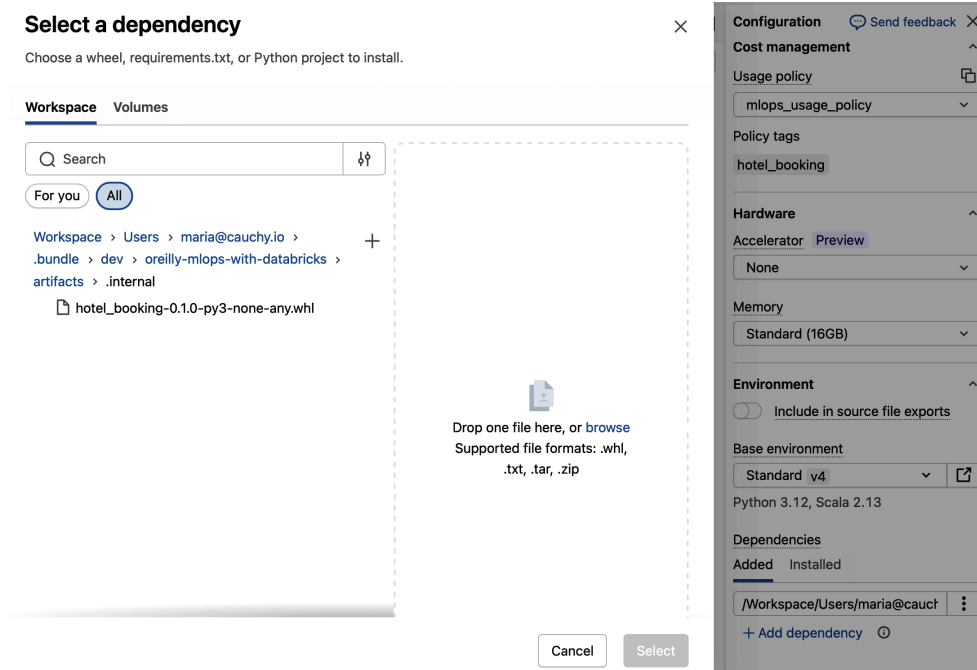


Figure 2-7. Serverless environment configuration

Since selecting an environment is a manual process, developers may forget to choose one. Therefore, you can configure a default custom base environment for serverless compute to ensure that the correct environment is applied to the project.

This environment will be automatically applied to all notebooks when serverless compute is selected. The following `env_v4.yaml` file will be uploaded to `/Volumes/mlops_dev/utils/envs/`:

```
environment_version: "4"
dependencies:
- mlflow==3.11.1
- numpy==2.4.0
- pandas==2.3.0
- pyarrow==22.0.0
- databricks-sdk==0.102.0
- scikit-learn==1.7.0
- lightgbm==4.6.0
- scipy==1.16.0
- pydantic==2.11.7
```

[Figure 2-8](#) demonstrates how a workspace admin (who will have the necessary permissions) can set this environment as the workspace base environment via the *Workspace Compute* settings. The *Compute* settings also allow you to configure a default package repository, which is useful when your organization uses [JFrog](#) or another private package repository.

Workspace base environments for serverless compute

Workspace base environments ensure that users automatically have a predefined environment available in their notebooks. [Documentation](#)

Create workspace base environment ✕

Base environment name

Base environment file path
 📁

☐ Build for Serverless GPU Compute Beta

Cancel Create

Figure 2-8. Setting a workspace base environment

Using all-purpose compute

If serverless compute does not satisfy your development needs, then you can always opt for all-purpose compute. Earlier in the chapter, we went through configuration of all-purpose compute. We can create a cluster with these specifications, meaning that we will be using Databricks Runtime 17.3 LTS (which is close to environment 4 in terms of its operating system, Python version, and installed packages).

Much like how you installed a custom library in the serverless environment, you can install it on an all-purpose cluster. The custom package wheel is built together with the bundle, and you can find it under the bundle root in the *artifacts/.internal* folder (see [Figure 2-9](#)).¹⁴

Install library 💬 Send feedback ✕

Library Source ⓘ
☒ Workspace ☐ Volumes ☐ File Path/ADLS ☐ PyPI ☐ Maven

Workspace File Path
Choose from workspace or upload a whl/wheelhouse.zip/requirements.txt file to install as a library

For you All

[Workspace](#) > [Users](#) > [maria@cauchy.io](#) > [.bundle](#) > [dev](#) > [oreilly-mlops-with-databricks](#) > [artifacts](#) > [.internal](#)

☒ 📁 hotel_booking-0.1.0-py3-none-any.whl

Figure 2-9. Installing a custom library on an all-purpose compute

If the custom package has dependencies hosted in a private package repository, you need to make sure that the default package repository is configured in the Compute settings.

We have already gone through the options for making sure that custom dependencies are available in the notebook for both serverless and all-purpose compute options. However, this is not an interactive way to develop, especially if you want to change the modules in your package. Also, we have assumed that we've been using the bundle deploy option—but what if we used Git folders instead?

To install specific libraries across all nodes of a cluster from a Databricks notebook, you can also use the `%pip install` command. This allows you to install them from PyPi, a private repository, or a wheel located in the workspace or a volume.

You can also install all the project dependencies specified in the *pyproject.toml* file by running the `pip install -e .` command and restarting Python with the `dbutils.library.restartPython()` command. To make the project itself importable, you also need to run the following code to adjust the path, which is considered a bad development practice:

```
from pathlib import Path
import sys

sys.path.append(str(Path.cwd().parent / 'src'))
```

However, this is necessary in this example because our package code is under the *src* folder.

Overall, developing directly in a Databricks notebook is not my first choice. Notebooks make it difficult to structure code into modules, run tests, and apply consistent formatting. However, to be 100% sure that the code works as intended, you ultimately need to execute it in a Databricks environment.

However, you cannot fully reproduce the same environment you have on Databricks locally—you can only achieve an approximation (as we did earlier). This is because the Databricks Runtime is curated and preconfigured with a set of installed libraries. When additional dependencies are installed on top of it, the resulting dependency tree may differ from what

you have locally. Installing all runtime libraries locally would not solve this problem either, as the requirements are not OS-agnostic.

The closest you can get locally is by using a Databricks Runtime image in combination with development containers. Even then, the Python environment will not be identical, as Databricks adds additional components that cannot be reproduced outside of the platform.

Fortunately, Databricks developer tools are quite mature today. As an alternative to running notebooks interactively, you can wrap a notebook as a job with specified compute and dependencies and execute it using the `databricks bundle run <job-resource-name>` command. We will go through this approach in [Chapter 5](#) (for classical ML) and [Chapter 9](#) (for LLM-based applications).

Conclusion

In this chapter, we went through the cluster configurations, the project setup, best practices, and different ways of developing on Databricks. We also showed you some alternatives to developing in a Databricks notebook. But even if you use notebooks, there are some steps you can take that will reduce time to production in the future:

- Refactor your code into functions and create classes. Structuring code using classes can make the codebase more organized, readable, and modular. With classes, you can bundle class attributes and methods (class functions) into a single unit. You should also separate the data loading, processing, and output steps. This will make the code easier to test and modify.
- Move code to separate modules. Create a new module for each set of classes. This will make your codebase more modular and reusable. It will also keep your repository nice and tidy and well organized.
- Package your code. Treat every Python project as a package and specify all the project dependencies and tool configurations in the *pyproject.toml* file.
- Create a configuration file or module (like the *project_config.yaml* file) to store hyperparameters and settings. Keeping configuration settings separate from the main code will allow you to easily adjust the parameters without modifying the main code. It will also be easier to create test scenarios with different configuration settings.
- Add logging to provide a detailed record of what happens during the execution of your program. When errors or unexpected behavior oc-

cur, these logs can help you diagnose and debug issues.

- Test your code to “fail fast” so you can avoid facing issues in the middle of running your code.

1 You can download the dataset from [Kaggle](#).

2 You can read more about Anaconda on [their official website](#).

3 Poetry did better than other tools, became one of the most popular backends in the ML community, and contributed to the wider adoption of *pyproject.toml*. You can read a blog about the [popularity of Python build backends over time](#).

4 As of 2026, the company behind uv, Astral, has been acquired by OpenAI. Although it has committed to continue support, if anything changes, the *pyproject.toml* standard we follow in this book should ensure stability for other package managers if required.

5 Note that we use the “`overwrite`” mode here as we are dealing with a static dataset.

6 For more details, consult [the Databricks documentation](#).

7 Consult the Databricks documentation at <https://oreil.ly/6gL1u> for all supported DABs resources.

8 Consult the Databricks documentation at <https://oreil.ly/qSSdT> for more details on development mode.

9 You can also define the `workspace` configuration at the top level of the file, but it’s not done much in practice.

10 DABs were originally built on top of [the Databricks Terraform provider](#). Databricks CLI version 0.279.0 and above support two different deployment engines: *Terraform* and *direct*. The Terraform deployment engine will eventually be deprecated, and the direct deployment engine will become the default thereafter. It uses [Databricks Go SDK](#).

11 To install the extension, follow [the official documentation](#).

12 You can find Python Env in [the Visual Studio Code marketplace](#).

13 This works because the extension created a configuration in the `.vscode` folder (`settings.json`).

14 Given that we also synchronized the *dist* folder in the bundle configuration, we could reference the wheel from the *dist* folder instead.

Previous chapter

< [1. MLOps Principles and Components](#)

Next chapter

[3. MLflow for Traditional ML](#) >